



CD SECURITY

For the Few Who Demand Perfection

AUDIT REPORT

HyperLend - Aviya
March 2026

Prepared by
Mr Auditor
Pelz
noman

Introduction

A time-boxed security review of the **HyperLend** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About HyperLend

Aviya is HyperLend's private KYC/KYB-gated lending market built on the Aave V3 codebase. These periphery contracts handle facility creation, token deployment, transfer whitelisting, and per-user position management.

Aviya enables highly customized, permissioned loans between counterparties. Each facility is an isolated lending arrangement with its own set of tokens and whitelist rules:

- Wrapper tokens represent a deposited underlying asset (1:1). The same underlying (e.g. USDC) can appear in many facilities simultaneously with different terms.
- Mintable tokens represent off-chain real-world assets (RWAs) minted by a trusted custodian.
- All tokens enforce a transfer whitelist: only KYC/KYB-approved addresses may send or receive them.
- Tokens are grouped into e-modes in the lending pool so collateral and debt are isolated per facility.

Users can deploy personal **UserFacilityManager** contracts to hold one facility position per address (a single address may only participate in one facility at a time).

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

review commit hash - [4a5446385c668cf648fd662337f4401c93de9291](#)

fixes review commit hash - [df85df18b1385657d1b577153fe895fd42c79b1f](#)

Scope

The following smart contracts were in scope of the audit:

- `contracts/TransferWhitelistManager.sol`
- `contracts/tokenization/FacilityMintableToken.sol`
- `contracts/tokenization/FacilityWrapperToken.sol`
- `contracts/UserFacilityManager.sol`

The following number of issues were found, categorized by their severity:

- Critical & High: 0 issues
- Medium: 0 issues
- Low: 1 issues
- Informational: 9 issues

Findings Summary

ID	Title	Severity	Status
[L-01]	<code>FacilityWrapperToken</code> constructor mints unbacked seed shares, allowing extraction of first depositor underlying	Low	Acknowledged
[I-01]	<code>updateSeedFundsReceiver</code> does not super-whitelist the new receiver atomically	Informational	Fixed
[I-02]	Missing zero-amount validation in <code>burn</code> allows meaningless state transitions and event emission	Informational	Fixed
[I-03]	Missing <code>msg.value</code> validation in <code>executeCall</code> and <code>executeMultiCall</code> enables unintended spending of contract balance	Informational	Acknowledged
[I-04]	<code>UserFacilityManager.executeCall</code> can be marked <code>external</code> instead of <code>public</code>	Informational	Fixed
[I-05]	<code>WHITELIST_REMOVER_ROLE</code> is defined but never granted at deployment	Informational	Fixed

ID	Title	Severity	Status
[I-06]	<code>FacilityWrapperToken.unwrap</code> emits requested amount instead of delivered amount for fee-on-transfer underlyings	Informational	Fixed
[I-07]	<code>UserFacilityManager.allowRevert</code> parameter is semantically inverted and silently swallows failures inside <code>executeMultiCall</code>	Informational	Acknowledged
[I-08]	Constructor auto-whitelist of <code>seedFundsReceiver</code> emits no <code>SuperWhitelistUpdated</code> event	Informational	Fixed
[I-09]	<code>updateSeedFundsReceiver</code> does not emit an event	Informational	Fixed

Detailed Findings

[L-01] `FacilityWrapperToken` constructor mints unbacked seed shares, allowing extraction of first depositor underlying

Severity

Impact: Low

Likelihood: Low

Description

The `FacilityWrapperToken` constructor mints `1e6` wrapper shares to the `seedFundsReceiver` without requiring any matching underlying deposit:

```
_mint(
    transferWhitelistManager.seedFundsReceiver(),
    1e6
);
```

This creates wrapper shares that are not backed by underlying tokens. At deployment, the wrapper starts with a positive `totalSupply` while the contract holds zero underlying, breaking the one-to-one accounting relationship between wrapper shares and underlying tokens that `wrap()` / `unwrap()` is otherwise designed around:

```
totalSupply = 1e6
underlying.balanceOf(address(this)) = 0
```

When the first real user wraps underlying tokens, the wrapper receives the deposited underlying and mints corresponding shares. The contract state becomes:

```
totalSupply = 1e6 + userDeposit
underlying.balanceOf(address(this)) = userDeposit
```

The `seedFundsReceiver` can then unwrap the unbacked `1e6` shares and receive underlying tokens funded entirely by the first depositor:

```
function unwrap(uint256 amount) external {
    _burn(msg.sender, amount);
    underlying.safeTransfer(msg.sender, amount);
}
```

The seeded shares are treated as valid redeemable shares despite having no corresponding underlying deposit. After this extraction, the first depositor no longer has enough underlying in the wrapper contract to fully redeem their shares — their full unwrap reverts due to insufficient underlying balance.

The extractable value depends on the decimals of the underlying token. For a 6-decimal stablecoin such as USDC, the extractable value is 1 USDC per facility. For an 8-decimal token such as WBTC, the extractable value is 0.01 WBTC. For 18-decimal assets, the value is dust. For 0-decimal assets, the seeded amount can represent the full first-depositor balance.

Recommendations

Either (a) remove the seed mint entirely, or (b) require the seed receiver to deposit matching underlying in the same transaction before minting the seed shares, or (c) mint the seed shares to `address(0)` (or another permanently inaccessible address) so they cannot be redeemed against future user deposits.

If the seed must remain redeemable, enforce matched backing:

```
underlying.safeTransferFrom(
    transferWhitelistManager.seedFundsReceiver(),
    address(this),
    1e6
);

_mint(
    transferWhitelistManager.seedFundsReceiver(),
    1e6
);
```

Client

Acknowledged - Intended design choice. Each market is seeded with at least 1e4 units which are then sent to `0xdead`; the seed mint exists so deployments do not have to burn real funds. Since admin is trusted, this is not considered an issue.

[I-01] `updateSeedFundsReceiver` does not super-whitelist the new receiver atomically

Description

The `TransferWhitelistManager` constructor correctly initializes the `seedFundsReceiver` and simultaneously grants the receiver super-whitelist privileges, guaranteeing that the initial receiver can freely transfer the seeded tokens immediately after deployment:

```
constructor(address newOwner, address newSeedFundsReceiver) {  
    ...  
    seedFundsReceiver = newSeedFundsReceiver;  
    superTransferWhitelist[newSeedFundsReceiver] = true;  
}
```

However, `updateSeedFundsReceiver` updates only the receiver pointer and does not touch the super-whitelist:

```
function updateSeedFundsReceiver(address newSeedFundsReceiver) external  
onlyRole(DEFAULT_ADMIN_ROLE) {  
    require(newSeedFundsReceiver != address(0), "TransferWhitelistManager:  
zero newSeedFundsReceiver");  
    seedFundsReceiver = newSeedFundsReceiver;  
    // super-whitelist not touched  
}
```

After admin rotation, newly deployed facility tokens mint their 1e6 seed allocation directly to the new receiver via `_mint(transferWhitelistManager.seedFundsReceiver(), 1e6)`. `_mint` bypasses whitelist enforcement, so the mint succeeds — but the new receiver cannot subsequently transfer the seed tokens. `transfer()` invokes `_verifyIsUserApproved(owner) → isUserApproved(token, newReceiver) → false || false → reverts`. The seed is effectively frozen until the admin separately calls `updateSuperWhitelist([newReceiver], true)`.

Admins already hold (or can grant) `SUPER_WHITELIST_ADMIN_ROLE`, so they can always follow up with a second transaction — no funds are lost or permanently locked. The observation is an ergonomic / consistency one: the constructor establishes the invariant "`seedFundsReceiver` is super-whitelisted" atomically in a single call, but the rotation path requires two separate transactions and leaves a window in between where the invariant does not hold and any fresh token deployment would mint its seed into a non-transferable address.

Recommendations

Mirror the constructor's write inside `updateSeedFundsReceiver` so a single admin transaction preserves the invariant and emits the corresponding event:

```
function updateSeedFundsReceiver(address newSeedFundsReceiver) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(newSeedFundsReceiver != address(0), "TransferWhitelistManager:
zero newSeedFundsReceiver");
    address oldSeedFundsReceiver = seedFundsReceiver;
    seedFundsReceiver = newSeedFundsReceiver;
    if (!superTransferWhitelist[newSeedFundsReceiver]) {
        superTransferWhitelist[newSeedFundsReceiver] = true;
        emit SuperWhitelistUpdated(newSeedFundsReceiver, true);
    }
    emit SeedFundsReceiverUpdated(oldSeedFundsReceiver,
newSeedFundsReceiver);
}
```

Client

Fixed.

[I-02] Missing zero-amount validation in `burn` allows meaningless state transitions and event emission

Description

The `burn` function in both `FacilityMintableToken.sol` and `FacilityWrapperToken.sol` accepts a user-supplied `amount` parameter but fails to validate that the value is greater than zero before executing the burn logic:

```
function burn(uint256 amount) external {
    if (isWhitelistEnabled){
        _verifyIsUserApproved(msg.sender);
    }

    _burn(msg.sender, amount);
    emit Burn(msg.sender, amount);
}
```

When `amount` is zero, the function still performs the whitelist verification and emits a `Burn` event even though no tokens are actually burned. This creates unnecessary state transitions and produces misleading on-chain signals that suggest a real token burn occurred. The absence of a basic sanity check also violates expected input-validation standards for token accounting functions — burn operations are intended to represent a meaningful reduction in supply, and accepting zero-value operations undermines that invariant.

Recommendations

Add an explicit validation that rejects zero-value burn requests before executing any logic:

```
function burn(uint256 amount) external {
    require(amount > 0, "Burn amount must be greater than zero");

    if (isWhitelistEnabled){
        _verifyIsUserApproved(msg.sender);
    }

    _burn(msg.sender, amount);
    emit Burn(msg.sender, amount);
}
```

Client

Fixed.

[I-03] Missing `msg.value` validation in `executeCall` and `executeMultiCall` enables unintended spending of contract balance

Description

The `executeCall` and `executeMultiCall` functions are declared `payable` but fail to validate that `msg.value` matches the ETH value being forwarded in each call. Both functions pass the provided `value` parameter directly into `_executeCall`, which executes an external call using the contract balance:

```
function executeCall(
    address target,
    uint256 value,
    bytes memory data,
    bool allowRevert
) public payable onlyOwner() returns (bytes memory) {
    return _executeCall(target, value, data, allowRevert);
}
```

In Solidity, ETH forwarded through `call` is deducted from `address(this).balance` rather than strictly from `msg.value`. Any ETH already stored in the contract can be used to fund calls even when the caller sends no ETH. An owner can execute the following scenario:

- The contract holds 1 ether from a previous refund or direct transfer.
- The owner calls `executeCall` with `value = 1 ether` and `msg.value = 0`.
- The call succeeds and transfers 1 ether from the contract balance to the target address.

This transfer occurs without any validation that the current transaction funded the call. The same issue exists in the batch execution path:

```
function executeMultiCall(Call[] memory calls) external payable onlyOwner()
{
    for (uint256 i = 0; i < calls.length; i++){
        _executeCall(
            calls[i].target,
            calls[i].value,
            calls[i].data,
            calls[i].allowRevert
        );
    }
}
```

If `msg.value` is less than the total required value, later calls in the batch can fail or be skipped depending on the `allowRevert` flag, creating inconsistent execution and leaving approvals, transfers, or state changes partially completed.

Recommendations

Add explicit validation to ensure that the ETH supplied in the transaction matches the ETH required for execution:

```
function executeCall(
    address target,
    uint256 value,
    bytes memory data,
    bool allowRevert
) public payable onlyOwner() returns (bytes memory) {
    require(msg.value == value, "msg.value must equal value");
    return _executeCall(target, value, data, allowRevert);
}

function executeMultiCall(Call[] memory calls) external payable onlyOwner()
{
    uint256 totalValue;
    for (uint256 i = 0; i < calls.length; i++) {
        totalValue += calls[i].value;
    }
    require(msg.value == totalValue, "msg.value must equal total call value");

    for (uint256 i = 0; i < calls.length; i++) {
        _executeCall(
            calls[i].target,
            calls[i].value,
            calls[i].data,
            calls[i].allowRevert
        );
    }
}
```

```
}  
}
```

Client

Acknowledged - Intended design choice. The protocol wants to allow maximum flexibility to the user, so they can either spend native tokens held in the contract or funds forwarded in `msg.value`.

[I-04] `UserFacilityManager.executeCall` can be marked `external` instead of `public`

Description

`executeCall` is declared `public`:

```
function executeCall(  
    address target,  
    uint256 value,  
    bytes memory data,  
    bool allowRevert  
) public payable onlyOwner() returns (bytes memory) {  
    return _executeCall(target, value, data, allowRevert);  
}
```

There is no internal caller in the contract — `executeMultiCall` invokes `_executeCall` directly rather than going through `executeCall`:

```
function executeMultiCall(Call[] memory calls) external payable onlyOwner()  
{  
    for (uint256 i = 0; i < calls.length; i++){  
        _executeCall(calls[i].target, calls[i].value, calls[i].data,  
calls[i].allowRevert);  
    }  
}
```

`public` is therefore marginally more gas per call (memory vs calldata for `data`) and weakens the signal that this function is strictly an EOA entry point.

Recommendations

Mark `executeCall` as `external` and change `bytes memory data` to `bytes calldata data` (forward-compatible with `_executeCall` since it still takes `bytes memory`):

```
function executeCall(
    address target,
    uint256 value,
    bytes calldata data,
    bool allowRevert
) external payable onlyOwner() returns (bytes memory) {
    return _executeCall(target, value, data, allowRevert);
}
```

Client

Fixed.

[I-05] **WHITELIST_REMOVER_ROLE** is defined but never granted at deployment

Description

`TransferWhitelistManager` declares three roles — **SUPER_WHITELIST_ADMIN_ROLE**, **WHITELIST_ADDER_ROLE**, **WHITELIST_REMOVER_ROLE** — but only two are assigned in the constructor:

```
constructor(address newOwner, address newSeedFundsReceiver) {
    ...
    _grantRole(DEFAULT_ADMIN_ROLE, newOwner);
    _grantRole(SUPER_WHITELIST_ADMIN_ROLE, newOwner);
    _grantRole(WHITELIST_ADDER_ROLE, msg.sender);           // factory

    seedFundsReceiver = newSeedFundsReceiver;
    superTransferWhitelist[newSeedFundsReceiver] = true;
}
```

Until a post-deploy transaction grants **WHITELIST_REMOVER_ROLE** to some address, no one can call `removeFromWhitelist` — so an address added to a per-token whitelist in error (e.g. a malicious user slipped past KYC, a typo, or a compromised counterparty) cannot be evicted. The only remedy during that window is `setEnableWhitelist(false)`, which disables all transfer restrictions globally for that token. That is too blunt for routine incident response, and is itself gated only by `superAdmin` with no timelock.

The role separation itself is intentional — the NatSpec flags that removal is expected to be a timelocked action ("in practice this should be a long & secure timelock (e.g. Timelock A in HyperLend governance)"). The issue is operational: nothing in the deployment path forces the role to be granted before the manager accepts users, so a facility can be live and accepting whitelisted users while still having no eviction path.

Recommendations

Pick one:

1. Accept a `whitelistRemover` address as a constructor argument and grant the role there. If the role holder is not known at deploy time, accept `address(0)` explicitly and revert any attempt to add users to the whitelist until it has been set.
2. Alternatively, add a deployment checklist item (in the README and in the factory's deploy script) that `WHITELIST_REMOVER_ROLE` MUST be granted to the timelock contract before the first facility is whitelisted, and guard the factory's `_setApprovedUsers` path behind a check that the role has been assigned.

Document whichever choice is made — the current implicit contract ("operator remembers to do it after deploy") is fragile.

Client

Fixed.

[I-06] `FacilityWrapperToken.unwrap` emits requested amount instead of delivered amount for fee-on-transfer underlyings

Description

Commit [4a54463](#) fixed `wrap()` to emit the balance delta rather than the requested amount so that the event is correct under fee-on-transfer underlyings. `unwrap()` was not symmetrically fixed:

```
_burn(msg.sender, amount);
underlying.safeTransfer(msg.sender, amount);
emit Unwrap(msg.sender, amount); // <-- requested amount, not delivered
amount
```

The contract explicitly tolerates FoT tokens on the wrap path ("we still check balance change just in case"). The asymmetry means any off-chain indexer that reconstructs a user's realised underlying position from `Wrap` / `Unwrap` logs will under-report fee drag when an FoT underlying is used. The on-chain accounting of `FacilityWrapperToken` itself is internally consistent — the bug is confined to event fidelity and tooling downstream of it.

Recommendations

Apply the same delta-accounting pattern to `unwrap()`:

```
function unwrap(uint256 amount) external nonReentrant {
    if (isWhitelistEnabled){
        _verifyIsUserApproved(msg.sender);
    }
}
```

```

    _burn(msg.sender, amount);

    uint256 balanceBefore = underlying.balanceOf(msg.sender);
    underlying.safeTransfer(msg.sender, amount);
    uint256 balanceAfter = underlying.balanceOf(msg.sender);

    emit Unwrap(msg.sender, balanceAfter - balanceBefore);
}

```

If the protocol explicitly does not want to support FoT underlyings, consider going the other direction — revert on any FoT detected in `wrap()` and document FoT as unsupported — which also removes the need for delta accounting entirely.

Client

Fixed.

[I-07] `UserFacilityManager.allowRevert` parameter is semantically inverted and silently swallows failures inside `executeMultiCall`

Description

`UserFacilityManager._executeCall` treats `allowRevert == true` as "swallow a failed low-level call and return the failed `returnData` as if it had succeeded":

```

(bool success, bytes memory returnData) = target.call{value: value}(data);
if (!allowRevert) {
    if (!success) _revertWithReason(returnData);
}
return returnData;

```

The parameter name reads in English as "this call is allowed to revert" — a caller composing a multicall would reasonably set `allowRevert: true` meaning "this sub-call may fail, that's fine, let it bubble up". What actually happens is the opposite: the failure is hidden, the outer batch continues, and subsequent `Call[]` entries execute on top of the stale state the failed step was supposed to set up. Callers have no indication that a step was skipped — there is no event and `executeCall` / `executeMultiCall` do not return a success flag. This is not an access-control bug (entry points are `onlyOwner`), but is a concrete footgun for DAO scripts, strategists, or smart-wallet owners composing batches offline.

Recommendations

Rename `allowRevert` → `allowFailure` (or `ignoreFailure`, `continueOnFailure`) in the struct, the external signature, the NatSpec, and the test descriptions. Alternatively, strengthen the NatSpec to spell out that the failure is swallowed silently and that chained side-effectful calls must not assume the

preceding step applied. Returning a success flag from `executeCall` / `executeMultiCall` would also let off-chain tooling distinguish swallowed failures from genuine successes.

Client

Acknowledged.

[I-08] Constructor auto-whitelist of `seedFundsReceiver` emits no `SuperWhitelistUpdated` event

Description

`TransferWhitelistManager`'s constructor writes `superTransferWhitelist[newSeedFundsReceiver] = true` directly, without going through `_setWhitelistStatus` or emitting the `SuperWhitelistUpdated` event that every other mutation of the map emits. An off-chain indexer that reconstructs super-whitelist membership from logs will show `seedFundsReceiver` as not super-whitelisted even though on-chain it is, until some later `updateSuperWhitelist` call happens to touch the same address.

Recommendations

Emit the event in the constructor so that every `superTransferWhitelist` entry has a corresponding log:

```
seedFundsReceiver = newSeedFundsReceiver;
superTransferWhitelist[newSeedFundsReceiver] = true;
emit SuperWhitelistUpdated(newSeedFundsReceiver, true);
```

Client

Fixed.

[I-09] `updateSeedFundsReceiver` does not emit an event

Description

`TransferWhitelistManager.updateSeedFundsReceiver` rotates a protocol-wide state variable — the recipient of every future token's 1e6-unit seed mint — without emitting any event. Every other privileged setter in the scoped code (`updateSuperWhitelist`, `_setWhitelistStatus`, `setMinter`, `setEnableWhitelist`, `SuperAdminUpdated`) emits a dedicated event. Off-chain monitoring and incident-response tooling cannot reliably detect the rotation.

Recommendations

Emit a dedicated event with both addresses indexed:

```
event SeedFundsReceiverUpdated(address indexed oldReceiver, address indexed newReceiver);

function updateSeedFundsReceiver(address newSeedFundsReceiver) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(newSeedFundsReceiver != address(0), "TransferWhitelistManager:
zero newSeedFundsReceiver");
    emit SeedFundsReceiverUpdated(seedFundsReceiver, newSeedFundsReceiver);
    seedFundsReceiver = newSeedFundsReceiver;
}
```

Client

Fixed.